

M7. Encapsulation



Ce module explore le concept essentiel de **l'encapsulation en Python 3**, un aspect fondamental de la programmation orientée objet. Les participants comprendront les différents avantages de l'encapsulation, tels que l'amélioration de la sécurité et la flexibilité du code. Le cours dévoile les différents niveaux d'accès aux données et explique comment utiliser les méthodes d'accès et de modification pour interagir avec les données de classe de manière sécurisée. De plus, nous explorons le décorateur `@property`, qui joue un rôle crucial dans la définition des getters et setters.



Les avantages de l'encapsulation



Les niveaux d'accès aux données



Les méthodes d'accès et de modification



Le décorateur `@property`



Review

Les avantages de l'encapsulation

Il est temps de se pencher plus profondément sur **le concept d'encapsulation** évoqué brièvement dans les leçons précédentes.



IMPORTANT

Il est important de comprendre que **l'encapsulation** est l'un des concepts fondamentaux de la POO. Il vise à **regrouper les données (attributs) et les méthodes (fonctions) qui les manipulent en une seule unité appelée classe**.

L'encapsulation est mise en œuvre grâce à la gestion des attributs au moyen de différents modificateurs de visibilité : *public*, *private* et *protected*.

Bien que la programmation orienté objet telle qu'elle est implémentée en Python a pleinement intégré la notion d'encapsulation. Il est fort d'admettre que la philosophie du langage, relativement proche du PHP en terme de permissivité, favorise de manière préférentielle l'accès aux attributs plutôt que leur stricte restriction et donc une utilisation forte et restrictive de l'encapsulation.

Vous allez d'ailleurs voir dans la prochaine leçon à quel point Python laisse la porte ouverte aux développeurs-euses quant à la modification de données supposément privées.

Qu'en est-il des avantages ?

Les avantages majeurs de l'encapsulation lors de son utilisation stricte (telle que recommandée dans la description du paradigme de programmation orienté objet) résultent dans la manière dont l'accès aux données d'une classe sera géré, ce qui aura pour effet de **favoriser la modularité** et **la réutilisation du code** ainsi que de **la maintenance** de celui-ci, d'**apporter une protection supplémentaire quant à l'accès aux données** (au sein du code, c'est-à-dire par les développeurs-euses) et la protection des données face à des modifications non désirées tout en permettant l'intégration de logiques de validation sur les données à modifier.

Ces principaux avantages résultant bien entendu du seul cas d'une application stricte de l'encapsulation et du respect strict des conventions liées à cette manière de travailler.

CONTINUER

Les niveaux d'accès aux données

L'encapsulation peut être assimilée à la manière dont nous allons gérer le mode d'accès aux données.

En Python, contrairement à d'autres langages qui en possèdent beaucoup plus, nous ne disposons que de **3 types d'encapsulation**, c'est-à-dire 3 niveaux d'accès aux données qui auront chacun leurs avantages.

Ces niveaux d'accès sont définis au moyen de procédés appelés **modificateurs de visibilité** : *public*, *private* et *protected*.

Le modificateur de visibilité *public*

Le modificateur *public* est sans le savoir le plus connu de tous. C'est en effet le modificateur de visibilité par défaut du langage.

Son rôle est de permettre l'**accès aux données tant à l'intérieur qu'à l'extérieur d'une classe**, sans aucune forme de restriction.

Ses principaux avantages sont :

- La facilité d'accès aux données depuis n'importe quel endroit du code.

- Son utilité lorsqu'il s'agit d'accéder à des données qui n'ont besoin d'aucune forme de protection ou de validation particulière.

Étant le modificateur par défaut, il ne dispose d'**aucune notation particulière**.

Voyons plutôt ce que donnerait une classe *Personne* définie au moyen de modificateurs de visibilité public :

```
class Personne:

    def __init__(self, nom, prenom):
        self.nom = nom;
        self.prenom = prenom;

    def __str__(self):
        return f"""Objet {type(self).__name__ }
        Valeur de l'attribut nom: {self.nom}
        Valeur de l'attribut prenom: {self.prenom}"""

#création d'un objet Personne
unePersonne = Personne("Jean", "Claude")

#affichage des informations de l'objet au moyen de la redéfinition
#de la méthode __str__
print(unePersonne)

#modification du prenom
unePersonne.prenom = "Yves"

#affichage des informations de l'objet modifié
print(unePersonne)
```

Script illustrant le fonctionnement de modificateurs *public*.

Le résultat produit par ce code sera le suivant :

```
Objet Personne  
Valeur de l'attribut nom: Jean  
Valeur de l'attribut prenom: Claude
```

```
Objet Personne  
Valeur de l'attribut nom: Jean  
Valeur de l'attribut prenom: Yves
```

Résultat du script précédent.

Remarquez qu'aucun obstacle ne s'est présenté à l'exécution de ce code. Les attributs de la classe étant tous publics, nous étions totalement libre de les modifier à notre guise.

CONTINUER

Le modificateur de visibilité *private*

Le modificateur de visibilité *private* est le plus restrictif de tous.

Son rôle est de permettre l'**inaccessibilité totale d'un attribut** en dehors de la classe qui le définit ; donc de protéger l'attribut de toutes modifications extérieures à la classe.

Ses principaux **avantages** sont :

- Renforcer l'encapsulation en empêchant l'accès direct aux données internes.
- Permettre l'imposition d'une logique de validation ou de traitement de l'information lors de l'accès à ces données.

Pour définir un attribut privé, il nous faut obligatoirement commencer le nom de l'attribut par un double underscore (`__`), par exemple : `self.__nom_de_l_attribut`.

Voyons ce que donnerait une classe *Personne* définie exclusivement au moyen de modificateurs de visibilité *private* :

```
class Personne:
    def __init__(self, nom, prenom):
        self.__nom = nom;
        self.__prenom = prenom;

    def __str__(self):
        return f"""Objet {type(self).__name__ }
Valeur de l'attribut nom: {self.__nom}
Valeur de l'attribut prenom: {self.__prenom}"""

#création d'un objet Personne
unePersonne = Personne("Jean", "Claude")

#affichage des informations de l'objet
print(unePersonne)

#affichage de l'attribut prénom de l'objet
print(unePersonne.__prenom)
```

Script illustrant le fonctionnement de modificateurs *private*.

Le résultat produit par ce code sera le suivant :

```
Objet Personne
Valeur de l'attribut nom: Jean
Valeur de l'attribut prenom: Claude
```



```
print(unePersonne.__prenom)
AttributeError: 'Personne' object has no attribute '__prenom'
```

Résultat du script précédent.

Remarquez que l'objet a correctement été créé au moyen du constructeur et que l'exécution de la méthode `__str__` lors de l'affichage de l'objet s'effectue sans encombres.

De fait, les attributs de la classe sont utilisés en interne à la classe, au sein de la redéfinition de la méthode `__str__()`.

Cependant, il ne nous est pas possible d'accéder à la valeur en lecture ou en écriture de l'attribut `__prenom`, celui-ci étant privé et donc inutilisable en dehors de la classe elle-même.

Si nous souhaitons interagir avec l'attribut *private*, il nous faudra obligatoirement passer par une méthode dédiée comme dans l'exemple ci-dessous :

```
class Personne:
    def __init__(self, nom, prenom):
        self.__nom = nom;
        self.__prenom = prenom;

    def __str__(self):
        return f"""Objet {type(self).__name__ }
Valeur de l'attribut nom: {self.__nom}
Valeur de l'attribut prenom: {self.__prenom}"""

    def lireNom(self):
        return self.__nom

    def ecrireNom(self, nom):
        self.__nom = nom

#création d'un objet Personne
unePersonne = Personne("Jean", "Claude")
```

```
#récupérer l'attribut __nom
print("Nom récupéré à l'extérieur de la classe :", unePersonne.lireNom())

#modifier l'attribut __nom
unePersonne.ecrireNom("Jeannine")

#affichage des informations de l'objet
print(unePersonne)
```

Script qui utilise une méthode pour accéder à des attributs privés.

Le résultat produit par ce code sera le suivant :

```
Nom récupéré à l'extérieur de la classe : Jean

Objet Personne
Valeur de l'attribut nom: Jean
Valeur de l'attribut prenom: Claude
```

Résultat du script précédent.

Mission accomplie, nous avons réussi à modifier un attribut privé au moyen de méthodes permettant une interaction, au sein de la classe , avec les attributs privés de cette dernière.

CONTINUER

Le modificateur de visibilité *protected*

Le modificateur de visibilité `protected` est le **modificateur de tous les compromis**.

Son rôle est de signifier de manière **non restrictive** l'inaccessibilité d'un attribut en dehors de la classe qui le définit tout en permettant néanmoins son utilisation.

Les attributs *protected* ne sont donc pas strictement privés. Il est admis qu'il est vivement déconseillé de travailler en direct avec ces attributs en dehors de leurs classes.

Son avantage majeur est :

- Le fait qu'il offre **un niveau intermédiaire d'encapsulation**, signalant aux autres développeurs·euses qu'il est préférable de ne pas accéder au dit attribut, tout en autorisant toujours sa modification si le développeur fait fi des recommandations générales associées à la déclaration et l'utilisation d'un tel attribut.

Pour définir un attribut *protected*, il nous faut obligatoirement commencer le nom de l'attribut par un simple underscore (`_`), par exemple: `self._nom_de_l_attribut`.

CONTINUER

Les méthodes d'accès et de modification

Lorsqu'il s'agit de manipuler un attribut privé d'une classe, il y a lieu de mettre en place une méthode d'accès à l'attribut, afin de pouvoir **récupérer sa valeur en dehors de la classe**, ainsi qu'une méthode de modification de la valeur de l'attribut si nous souhaitons pouvoir **changer sa valeur en dehors de la classe**.

L'avantage majeur de ces méthodes réside dans le fait qu'il est tout à fait possible d'y insérer une logique de validation propre.

Cette logique aura notamment pour objectif de permettre la mise en place de plages de valeurs et autres conditions d'admissibilité pour l'attribut qu'il cible.

Par exemple, lorsqu'on demande d'insérer votre âge dans un input, la méthode de modification de l'attribut *private* `__age` pourrait contenir un ensemble de vérifications permettant de s'assurer d'obtenir un âge réaliste et pas une valeur aberrante telle que -1000 ans ou +10000 ans.

Exemple dans lequel un utilisateur doit encoder son nom, son prénom ainsi que son âge :

```
class Personne:
    def __init__(self, nom="", prenom="", age=0):
        if nom: self.nom = nom
        if prenom: self.prenom = prenom
        if age: self.encoderAge(age)

    def __str__(self):
```

```

        return f""Objet {type(self).__name__ }
Valeur de l'attribut nom: {self.nom}
Valeur de l'attribut prénom: {self.prenom}
Valeur de l'attribut âge: {self.__age}"""

    #méthode de modification
    def encoderAge(self, age):
        if 0 < age < 100:
            self.__age = age
        else: raise Exception("L'âge doit être compris entre 1 et 99, réessaye")

    #méthode d'accès
    def lireAge(self):
        return self.__age

#définition d'une variable de contrôle de boucle
encodagePending = True
#création d'un objet Personne
unePersonne = Personne()

nom = input("Encode ton nom : ")
prenom = input("Encode ton prenom : ")
age = 0

#Boucle d'encodage de l'âge
while encodagePending:
    try:
        age = int(input("Encode ton âge : "))
        unePersonne.encoderAge(age)
        encodagePending = False
    except Exception as err:
        print(err)
else:
    unePersonne.nom = nom
    unePersonne.prenom = prenom
    #affichage des informations de l'objet
    print(unePersonne)

```

Script comportant des vérifications et une gestion d'erreur dans le cadre de la mise à jour d'attributs.

Ce code va demander à l'utilisateur d'entrer son nom et son prénom (qui sont des attributs public de la classe *Personne*), ainsi que son âge qui est un attribut privé sujet à validation (l'âge doit être compris entre 1 et 99 ans).

Voici un exemple d'exécution :

```
#Etape 1 : l'utilisateur entre ses données publiques
Encode ton nom : Cooper
Encode ton prénom : Sheldon

#Etape2: l'utilisateur doit entrer son âge et encode une valeur aberrante
Encode ton âge : 350

#Etape3: la logique de validation fait son travail et signale une erreur à
L'âge doit être compris entre 1 et 99, réessayez

#Etape4: l'utilisateur se corrige
Encode ton âge : 35

#Etape5: les données sont mises à jour dans l'objet Personne et celui-ci s
Objet Personne
Valeur de l'attribut nom: Cooper
Valeur de l'attribut prénom: Sheldon
Valeur de l'attribut âge: 35
```

Résultat du script précédent.

Remarquez que la logique de validation fonctionne parfaitement bien. L'utilisateur est contraint d'encoder une valeur correcte avant de pouvoir passer à l'étape suivante du code.

CONTINUER

Conventions de nommage

Vous avez probablement remarqué dans la leçon précédente ainsi que dans l'exemple ci-dessus que les méthodes d'accès et de modification des attributs privés peuvent être écrites n'importe comment. Entendez par là que vous pouvez leur donner le nom que vous voulez, tant que la méthode est amenée à manipuler en lecture ou en écriture un attribut privé de la classe. Il s'agira bel et bien d'une méthode d'accès/de modification.

Des conventions existent afin d'apporter de la clarté dans le code. Par ailleurs, ces conventions sont issues de la dénomination initiale et anglophone données à ces méthodes à savoir que **la méthode d'accès ou accesseur** s'appelle un **getter** et que **la méthode de modification ou mutateur** s'appelle quant à elle un **setter**.

Il est donc normal de retrouver les conventions de nommage suivantes :

- L'**accesseur** sera nommé: `get_{nom public de l'attribut}()`, par exemple `get_age()`.
- Le **mutateur** sera nommé: `set_{nom public de l'attribut}()`, par exemple `set_age()`.

Adaptons notre précédent exemple en conséquence :

```
class Personne:
    def __init__(self, nom="", prenom="", age=0):
        if nom: self.nom = nom
        if prenom: self.prenom = prenom
        if age: self.set_age(age)

    def __str__(self):
        return f"Objet {type(self).__name__ }
Valeur de l'attribut nom: {self.nom}
Valeur de l'attribut prénom: {self.prenom}
Valeur de l'attribut âge: {self.__age}"
```

```

#méthode de modification (ou SETTER)
def set_age(self, age):
    if 0 < age < 100:
        self.__age = age
    else: raise Exception("L'âge doit être compris entre 1 et 99, réessaie")

#méthode d'accès (ou GETTER)
def get_age(self):
    return self.__age

#définition d'une variable de contrôle de boucle
encodagePending = True
#création d'un objet Personne
unePersonne = Personne()

nom = input("Encode ton nom : ")
prenom = input("Encode ton prénom : ")
age = 0

#Boucle d'encodage de l'âge
while encodagePending:
    try:
        age = int(input("Encode ton âge : "))
        unePersonne.set_age(age)
        encodagePending = False
    except Exception as err:
        print(err)
else:
    unePersonne.nom = nom
    unePersonne.prenom = prenom
    #affichage des informations de l'objet
    print(unePersonne)

```

Script mettant en œuvre les conventions de nommage.

Le tour est joué!

Il n'y a plus qu'à voir maintenant comment faire en sorte de travailler ce concept de manière encore plus fluide en se débarrassant des contraintes liées à l'utilisation d'une fonction

spécifique et en les remplaçant par un mode de fonctionnement se rapprochant le plus possible du fonctionnement d'un attribut traditionnel : plus de parenthèses, plus de paramètres, une simple affectation rapide et directe.

CONTINUER

Le décorateur `@property`

Un décorateur est une fonction qui a pour finalité de **modifier le comportement d'autres fonctions**.

Ils sont utilisés sous forme d'annotation au-dessus de la définition de la fonction qu'ils ciblent au moyen de la syntaxe `@nom_du_décorateur`.

`@property`

Le décorateur `@property` va permettre de transformer la méthode de classe qu'il cible en attribut / propriété.

Ce décorateur est presque exclusivement utilisé pour gérer l'accès aux attributs privés de la classe en simplifiant l'utilisation d'un getter pour ces attributs.

L'utilisation de la méthode getter décorée de votre classe pourra dès lors se faire de la même manière que si vous manipulez un attribut de la classe.

Comment fonctionne le décorateur `@property`

1

On place le décorateur au-dessus de la méthode ciblée.

2

Lorsqu'on accède à la méthode depuis une instance de la classe, elle est utilisée comme s'il s'agissait d'un attribut direct de l'objet, cette méthode est transformée en propriété.

3

On ajoute au besoin une logique de calcul ou de validation à la méthode décorée.

4

Lorsqu'on accède à la propriété, la méthode associée est appelée automatiquement et vous récupérez la valeur calculée ou validée au sein de la méthode.

Reprenons notre exemple précédent et transformons l'ancienne méthode getter en propriété :

```
class Personne:
    def __init__(self, nom="", prenom="", age=0):
        if nom: self.nom = nom
        if prenom: self.prenom = prenom
        if age: self.set_age(age)

    def __str__(self):
        return f"Objet {type(self).__name__ }
Valeur de l'attribut nom: {self.nom}
Valeur de l'attribut prénom: {self.prenom}
Valeur de l'attribut âge: {self.__age}"

    #SETTER
    def set_age(self, age):
        if 0 < age < 100:
            self.__age = age
        else: raise Exception("L'âge doit être compris entre 1 et 99, réessaie")

    #GETTER
    @property
    def age(self):
```

```
        return self.__age

unePersonne = Personne("Jean", "Claude", 50)
print(unePersonne.age)
```

Utilisation d'un décorateur pour un getter.

 Remarquez que le nom de la propriété est conventionnellement identique au nom de l'attribut privé auquel elle est associée.

Nous obtenons donc une manière de récupérer la valeur d'un attribut privé, de manière rapide et fluide.

Une fois la propriété créée, il est extrêmement simple de définir un *setter* sur base de cette propriété.

On définira une fonction *setter* avec les mêmes conventions de nommage que pour la fonction *getter* décorée et nous décorerons la fonction au moyen d'une extension du décorateur `@property` permettant de spécifier un *setter* à ajouter au *getter* précédemment créé.

La syntaxe du setter sera la suivante `@NomDeLaPropriété.setter`.

Ajoutons un setter sur notre dernier exemple :

```
class Personne:
    def __init__(self, nom="", prenom="", age=0):
        if nom: self.nom = nom
        if prenom: self.prenom = prenom
```

```

        if age: self.age = age

    def __str__(self):
        return f"Objet {type(self).__name__ }
Valeur de l'attribut nom: {self.nom}
Valeur de l'attribut prénom: {self.prenom}
Valeur de l'attribut âge: {self.__age}"

    #GETTER
    @property
    def age(self):
        return self.__age

    #SETTER
    @age.setter
    def age(self, age):
        if 0 < age < 100:
            self.__age = age
        else: raise Exception("L'âge doit être compris entre 1 et 99, réessaye")

unePersonne = Personne("Jean", "Claude", 50)
print(unePersonne.age)
unePersonne.age = 45
print(unePersonne.age)

```

Utilisation d'un décorateur pour un setter.

 Notez un élément primordial dans l'utilisation des getter/setter au moyen du décorateur `@property`: la propriété doit être déclarée AVANT le setter. Si ce n'est pas le cas, votre code génèrera une erreur.

CONTINUER

Review

Quels sont les avantages de l'encapsulation ?

- ☐ Faciliter la lecture du code
- ☐ Sécurisation de l'accès aux données
- ☐ Amélioration de la modularité du code
- ☐ Amélioration de la réutilisation du code

SUBMIT

Associez le modificateur de visibilité avec sa définition brève correspondante.

SUBMIT

Soit la classe reprise initialement dans le trinket ci-dessous.

Appliquez les règles d'encapsulation pour rendre tous les attributs de la classe *private* en créant également les *getters* et *setters* associés au moyen du décorateur *@property*.

Adaptez l'utilisation de l'objet en fonction de vos modifications.

3

Python3

< >

trinket

main.py solution.py

+ ↕ 📎

Choose File

No file chosen

Run

Share

Remix

CONTINUER